Inheritance

Introduction

- The capability of a class to derive properties and characteristics from another class is called Inheritance.
- Promotes ‘code reusability’.
- Superclass (Parent class): The class from which properties and methods are inherited.
- Subclass (Child class): The class that inherits the properties and methods from the superclass.
- A subclass inherits all members of its superclass, but it cannot access members that are declared as *private* in the superclass.
- Syntax:

```
class DerivedClass extends BaseClass
{
        //methods and fields
}
```

Example

```
class A {
Protected int number;
    public void display() {
        System.out.println("This is the display method of the superclass.");
    }
}
class B extends A {
    String message;
    public void show() {
        System.out.println("This is the show method of the subclass.");
    }
}
public class Main {
    public static void main(String[] args) {
B obj = new B();
        obj.display(); // Inherited from superclass obj.show();    // Defined in
subclass
    }   }
```

Access modifier for inheritance

1. private:

- The member is **accessible only within the same class**.
- It cannot be accessed by subclasses, even through inheritance.

2. default (package-private):

- If no modifier is specified, it is **accessible only within the same package**.
- It cannot be accessed by subclasses in a **different package**, even if inherited.

3. protected:

- The member is accessible within the **same package** (like default).
- It is also accessible by **subclasses**, even if they are in **different packages**.
- However, subclasses in different packages can access the protected members **only through inheritance**, not directly via an object reference.

4. public:

- The member is accessible **everywhere**—within the same class, package, subclass, or any other class.

```
class Animal {
    private String type = "Mammal"; // private: Not accessible to subclass
    String habitat = "Land";       // default: Accessible only within the same package
    protected String name = "Animal"; // protected: Accessible within subclass (even in different packages)
    public int age = 5;             // public: Accessible everywhere
    private void privateMethod() {
        System.out.println("Private method in Animal");
    }
    protected void protectedMethod() {
        System.out.println("Protected method in Animal");
    }
}
class Dog extends Animal {            // Subclass in the same package
    public void showInfo() {
        // System.out.println(type); // ERROR: private member not accessible
        System.out.println(habitat); // Accessible: default member (same package)
        System.out.println(name);    // Accessible: protected member
        System.out.println(age);     // Accessible: public member
        // privateMethod(); // ERROR: private method not accessible
        protectedMethod(); // Accessible: protected method
    }
}
```

Protected members are accessible in subclasses in different packages

```
// File: Animal.java
// Package: packageA
package packageA;
public class Animal {
    protected String name;
    public Animal(String name) {
        this.name = name;
    }
    protected void display() {
System.out.println("Animal name: " + name);
    }
}
```

```
// File: Main.java
// Package: packageB
package packageB;
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog("Buddy");
        dog.showDetails(); // Access protected members
    } }
```

```
// File: Dog.java
// Package: packageB
package packageB;
import packageA.Animal;
public class Dog extends Animal {
    public Dog(String name) {
this.name = name; // Accessing protected member directly
    }
    public void showDetails() {
        System.out.println("Dog name: " + name); // Accessing
protected member directly
        display(); // Calling protected method
    }
}
```

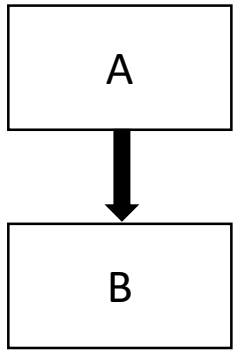
Types of Inheritance

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance

Single Inheritance

- When a class extends only one parent class.
- Example: class B extends A

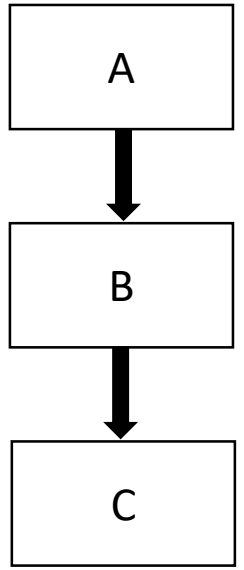
```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class TestInheritance{
public static void main(String args[]){
Dog d=new Dog();
d.bark();
d.eat();
}}
```



Multilevel Inheritance

- When there is a chain of inheritance, it is known as *multilevel inheritance*.

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class BabyDog extends Dog{
void weep(){System.out.println("weeping...");}
}
class TestInheritance2{
public static void main(String args[]){
BabyDog d=new BabyDog();
d.weep();
d.bark();
d.eat();
}}
```



Order of Execution of constructors

- When order of execution of constructors is from superclass to subclass, or base to derived class.

```
class Animal {
    public Animal() {
        System.out.println("Animal constructor called.");
    }
}
class Mammal extends Animal {
    public Mammal() {
        System.out.println("Mammal constructor called.");
    }
}
class Dog extends Mammal {
    public Dog() {
        System.out.println("Dog constructor called.");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog(); // Creating an object of Dog
    }
}
```

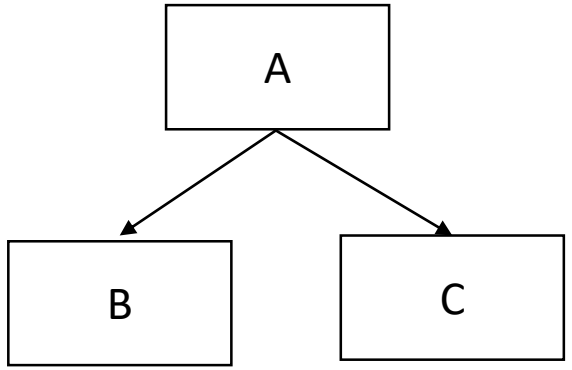
O/P:

```
Animal constructor called.
Mammal constructor called.
Dog constructor called.
```

Hierarchical Inheritance

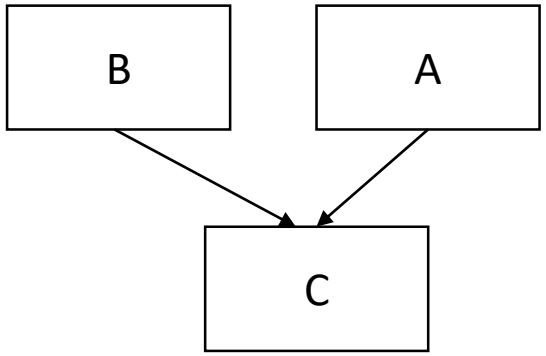
- When two or more classes inherits a single class.

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class Cat extends Animal{
void meow(){System.out.println("meowing...");}
}
class TestInheritance3{
public static void main(String args[]){
Cat c=new Cat();
c.meow();
c.eat();
//c.bark();//Error
}}
```



Multiple Inheritance

- A class can inherit from more than one superclass.
- **Java doesn’t support multiple inheritance with classes.**
- In Java, we can achieve multiple inheritances only through Interfaces.



Super keyword

- In Java, the super keyword denotes a reference variable that is used to refer to the immediate superclass of a class. It can be used to:
  - Access superclass methods and fields that are hidden by subclass members.
  - Call the superclass constructor to initialize the superclass part of an object.
- When calling a superclass constructor, the super() statement must be the first statement in the constructor of the subclass.

Use of super to access superclass Variables

```
class Vehicle {
        int maxSpeed = 120;
}
class Car extends Vehicle {
        int maxSpeed = 180;
        void display() {
                // print maxSpeed of base class (vehicle)
                System.out.println("Maximum Speed: "+ super.maxSpeed);
        }
}
class Test {
        public static void main(String[] args) {
                Car small = new Car();
                small.display();
        }
}
```

Use of super to access superclass Methods

```
class Person {
        void message()
        {
        System.out.println("This is person class\n");
        }
}
class Student extends Person {
        void message()
        {
        System.out.println("This is student class");
        }
        void display()
        {
        // invoke current  class message() method
                message();

        // will invoke class message() method
                super.message();
        }
}

class Test {
  public static void main(String args[])
  {
        Student s = new Student();

        // calling display() of Student
        s.display();
  }
 }
```

Use of super to access superclass constructors

```
class Person {
Person(){}

Person(int i) {
   System.out.println("Person class Constructor");
    }
}
class Student extends Person {
    Student() {
   // invoke or call parent class constructor
      super(10);
      System.out.println("Student class Constructor");
    }
}
class Test {
            public static void main(String[] args)
            {
                        Student s = new Student();
                                                                    }
                                                                }
```

Use of super to access superclass constructors

```
class Vehicle {
    protected String model; // Protected member
    protected int year;
    public Vehicle(String model, int year) {
        this.model = model;
        this.year = year;
}
 public void displayInfo() {
  System.out.println("Model: " + model + ", Year: " + year);
  }
}
class Car extends Vehicle {
    private String color; // Subclass member
public Car(String model, int year, String color) {
// Call the superclass constructor to initialize model and year
      super(model, year);
        this.color = color; // Initialize color
    }

    // Method to display car information
    public void displayCarInfo() {
        System.out.println("Car Color: " + color);
        displayInfo(); // Call superclass method to display model
and year
    }
}

public class SuperExample {
    public static void main(String[] args) {
        Car car = new Car("Toyota", 2022, "Red"); // Create a Car
object
        car.displayCarInfo(); // Call method to display car
information
    }
}
```

Order of execution

- When you use super() in the subclass constructor, it explicitly calls the constructor of the superclass. If super() is not explicitly called, Java automatically calls the no-argument constructor of the superclass (if available).

```
class Animal {
    public Animal(String name) {
    System.out.println("Animal constructor called. Name: " + name);
    }
}
class Mammal extends Animal {
    public Mammal(String name) {
        super(name); // Explicitly calling Animal constructor
        System.out.println("Mammal constructor called.");
    }
}
class Dog extends Mammal {
    public Dog(String name) {
        super(name); // Explicitly calling Mammal constructor
        System.out.println("Dog constructor called.");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog("Buddy"); // Creating an object of Dog
    }
}
```

O/P

Animal constructor called. Name: Buddy
Mammal constructor called.
Dog constructor called.

Real-life Example

```
class Vehicle {                 // Superclass: Vehicle
    protected String brand;
    protected int speed;
    public Vehicle(String brand, int speed) {
        this.brand = brand;
        this.speed = speed;
    }
public void displayInfo() {
        System.out.println("Brand: " + brand + " | Speed: " +
speed + " km/h");
    }
}
class Car extends Vehicle {             // Subclass: Car
    private int numDoors;
    public Car(String brand, int speed, int numDoors) {
        super(brand, speed); // Call the constructor of the
superclass
        this.numDoors = numDoors;
    }
public void displayCarDetails() {
        displayInfo(); // Reuse the superclass method
        System.out.println("Number of doors: " + numDoors);
    }
}
```

```
class Bike extends Vehicle {       // Subclass: Bike
    private boolean hasCarrier;

    public Bike(String brand, int speed, boolean hasCarrier) {
        super(brand, speed); // Call the constructor of the
superclass
        this.hasCarrier = hasCarrier;
    }
public void displayBikeDetails() {
        displayInfo(); // Reuse the superclass method
        System.out.println("Has carrier: " + (hasCarrier ? "Yes" :
"No"));
    }
}

public class Main {
public static void main(String[] args) {

Car car = new Car("Toyota", 180, 4);
car.displayCarDetails();
System.out.println();
Bike bike = new Bike("Yamaha", 120, true);
bike.displayBikeDetails();
}
}
```

Final keyword

- Final keyword is a modifier that can be applied to variables, methods, and classes to indicate that they cannot be changed or overridden.
- When a variable is declared as final, it means that its value cannot be changed once it has been assigned.
  - final int x = 10;
  - x = 20; // This will cause a compilation error
- When a method is declared as final, it cannot be overridden by subclasses.
- When a class is declared as final, no other class can extend it.

Final Methods

```
class Parent {
    public final void display() {
        System.out.println("This is a final method.");
    }
}

class Child extends Parent {
    // This will cause a compilation error
    // public void display() {
    //     System.out.println("Trying to override final method.");
    // }
}

public class FinalMethodExample {
    public static void main(String[] args) {
        Child child = new Child();
        child.display(); // Calls the final method from the Parent class
    }
}
```

Final Classes

```
final class FinalClass {
    public void show() {
        System.out.println("This is a final class.");
    }
}

// This will cause a compilation error
// class SubClass extends FinalClass {
// }

public class FinalClassExample {
    public static void main(String[] args) {
        FinalClass obj = new FinalClass();
        obj.show();
    }
}
```

Polymorphism

- The word ‘polymorphism’ means ‘having many forms’.
- In Java, it refers to the ability of an object to take on multiple forms.
- Types:
  - Compile time Polymorphism
  - Run time Polymorphism

Compile time polymorphism

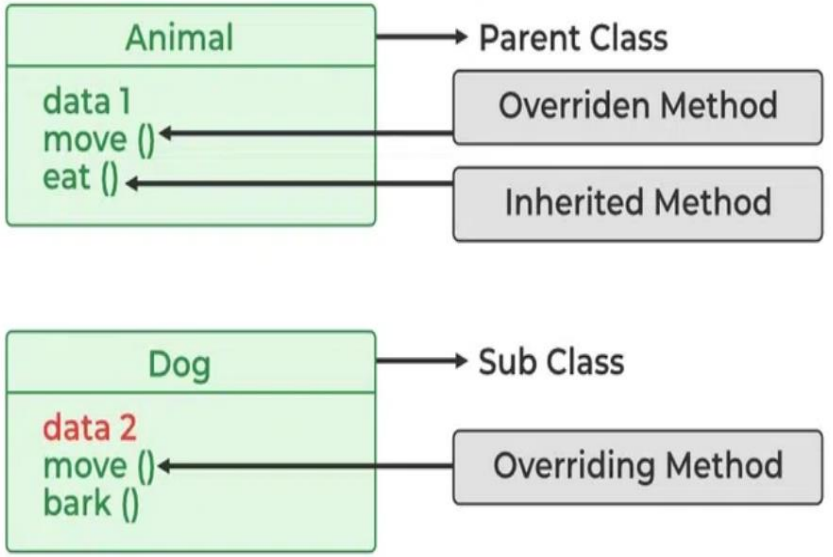
- It is also known as static polymorphism.
- This type of polymorphism is achieved by **function overloading** or operator overloading.
- Java does not support operator overloading.
- The method call is bound with method body at compile-time.

Run time polymorphism

- It is also known as Dynamic Method Dispatch.
- This type of polymorphism is achieved by **Method Overriding.**
- Method call is resolved at run time.
- Supports late binding.
- In runtime polymorphism, a subclass provides a specific implementation of a method that is already defined in its superclass.
- The method to be executed is determined at **runtime**, depending on the object type, even if the reference is of the superclass type.

Method Overriding

- Overriding is a feature that allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes.
- Used for run time polymorphism.



- Conditions:
  - The method in the subclass must have the same name, return type (or a covariant return type), and parameter list as the method in the superclass.
  - if there is a relationship through inheritance between the subclass and the superclass.
  - **Access Level**: The access level of the overriding method cannot be more restrictive than the overridden method in the superclass. For example, if the superclass method is public, the subclass method must also be public or less restrictive.

Method Overriding

- It is recommended (though not mandatory) to use the @Override annotation when overriding a method. This helps the compiler identify mistakes, like incorrect method signatures.
- **Static, Final, and Private Methods**: These methods cannot be overridden because static methods are bound at compile-time, and final and private methods are not inherited by subclasses.

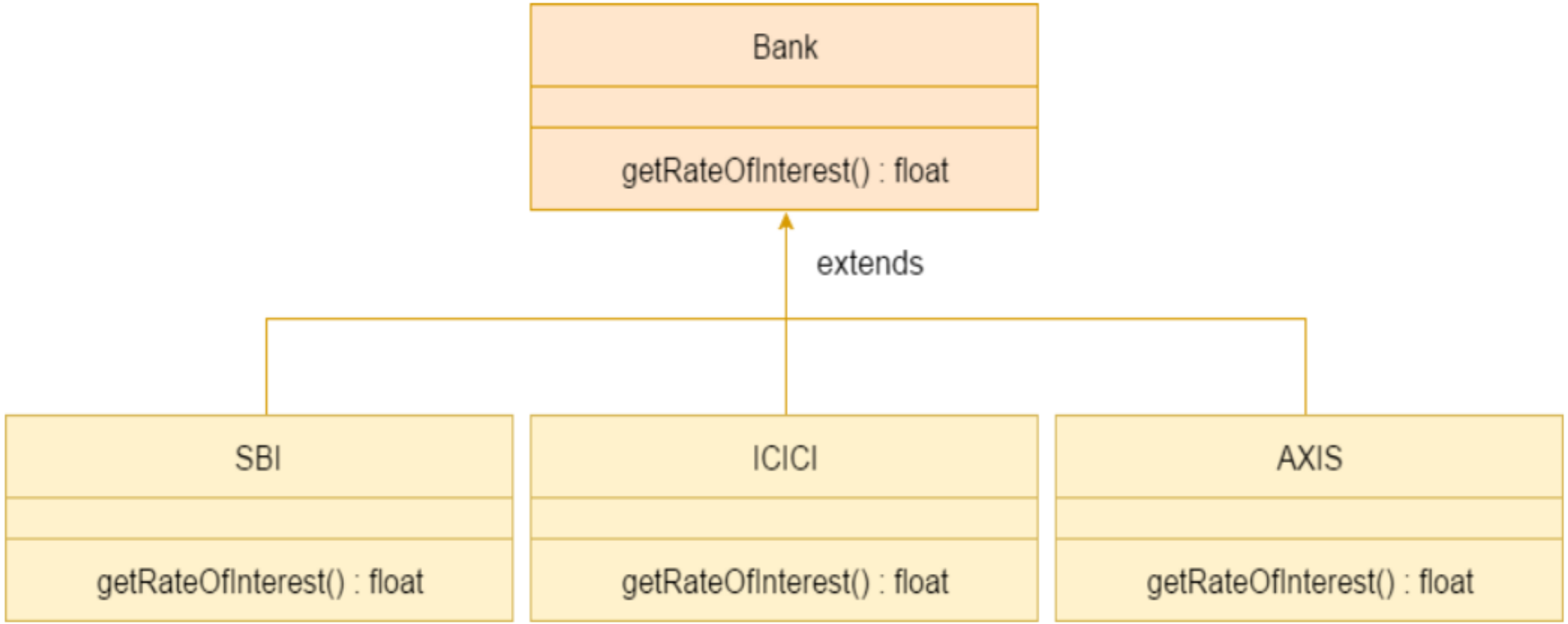
Example:

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {    //super.sound();
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog();
         myDog.sound();
}
}
```

A real-world example



Banking example

```
class Bank{
int getRateOfInterest(){return 0;}
}
//Creating child classes.
class SBI extends Bank{
int getRateOfInterest(){return 8;}
}

class ICICI extends Bank{
int getRateOfInterest(){return 7;}
}
class AXIS extends Bank{
int getRateOfInterest(){return 9;}
}
//Test class to create objects and call the methods
class Test{
public static void main(String args[]){
SBI s=new SBI();
ICICI i=new ICICI();
AXIS a=new AXIS();
System.out.println("SBI Rate of Interest: "+s.getRateOfInterest());
System.out.println("ICICI Rate of Interest: "+i.getRateOfInterest());
System.out.println("AXIS Rate of Interest: "+a.getRateOfInterest());
}
}
```

Method Overriding vs Method Hiding

- When you define a static method with the same signature as a static method in the base class, it is known as method hiding.

	Superclass Instance Method	**Superclass Static Method**
Subclass Instance Method	Overrides	Generates a compile-time error
Subclass Static Method	Generates a compile-time error	Hides

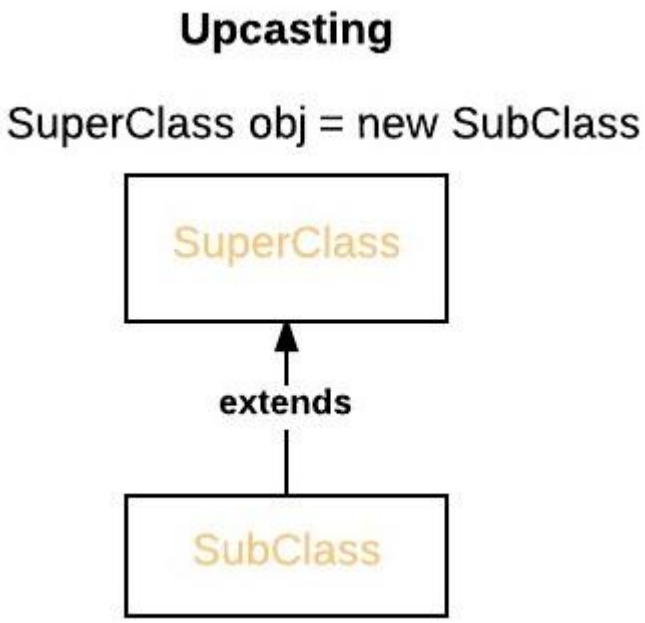
- Case: Static method in superclass, instance method in subclass
- Outcome: Compile time error.
- Why does this happen?
  - Static methods belong to the class, not to any instance. They are resolved at compile time.
  - Instance methods belong to the object (instance). They are resolved at runtime using dynamic method dispatch.
- So when the subclass tries to replace a static method with an instance method, Java doesn't know whether to resolve the method by class type (compile-time) or object type (runtime) — that would break the consistency of method binding. Hence, Java forbids this situation entirely.

Dynamic Method Dispatch

- Dynamic method dispatch is the mechanism by which calling of overridden method is resolved at run time, rather than compile time.
- It allows the program to choose which method to invoke based on the actual object's type, not the reference type.
- This only works for instance (non-static) methods.
- **Working**:
  - when a superclass reference variable is used to refer to a subclass object, the method that gets called is determined by the actual type of the object, not the reference type.
  - Dynamic Method Dispatch enables runtime method overriding, meaning if a subclass overrides a method in its superclass, the overridden method in the subclass will be called, even if the reference is of the superclass type.

Dynamic Method Dispatch

- A superclass reference variable can refer to a subclass object. This is also known as **upcasting**. Java uses this fact to resolve calls to overridden methods at run time.
- It is **automatic** and **implicit** in Java and does not require explicit casting.



Example (Upcasting)

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
public class Test {
        public static void main(String[] args) {
        Animal animal = new Dog();                    // Upcasting Dog to Animal
        animal.sound();                                   // Calls Dog's sound() method
       }
}
```

Downcasting

- Downcasting is the process of converting a superclass reference (or object) into a subclass reference.
- It’s the opposite of **upcasting** (which is automatic).
- It is explicit and requires casting because it's not always safe, and there’s a risk of a **ClassCastException** if done improperly.
- Downcasting is usually needed when you want to access subclass-specific methods or fields that are not accessible through the superclass reference.
- Syntax:
  - Parent p = new Child(); // upcasting
  - Child c = (Child)p; // downcasting

Downcasting

```
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void eat() {
        System.out.println("Dog is eating");
    }
 void bark(){
System.out.println("Dog barks");
}
}

public class Test {
    public static void main(String[] args) {
        Animal animal = new Dog();              // Upcasting
        animal.eat();                           //prints “Dog is eating’
    // animal.bark();                           //Error
        Dog dog = (Dog) animal;                 // Downcasting
        dog.bark();                             // Calls Dog's bark() method
    }
}
```

```
//Program: Run time polymorphism
class Phone{
  void alarm(){
    System.out.println("Alarm at 6 AM");
  }
  void call(){
    System.out.println("calling via phone");
  }
}
class Smartphone extends Phone{
  void clickPhoto(){
    System.out.println("click picture");
  }
   void call(){
    System.out.println("calling via smartphone");
  }
}
class User{
Phone ph;
public User(Phone ph){
this.ph=ph;
}
void user_call(){
ph.call();
}
void user_alarm(){
ph.alarm();
}
}

class Test{
public static void main(String args[]){
  Phone ph=new Phone();

  User ramesh=new User(ph);
  ramesh.user_call();
  ramesh.user_alarm();

}

}
```

Runtime Polymorphism with Data Members

In Java, we can override methods only, not the variables(data members),
so **runtime polymorphism cannot be achieved by data members.**

```
class A{
   int x = 10;
}
class B extends A{
   int x = 20;
}
public class Test{
   public static void main(String args[]){
      A a = new B(); // object of type B
System.out.println(a.x);                    //OP:10
   }
}
```

Abstract class

- An **abstract class** in Java is a class that cannot be instantiated on its own and serves as a blueprint for other classes.
- It may contain abstract methods (methods without a body) as well as non-abstract methods (methods with a body).
- Abstract classes can have constructors, but these are called when a subclass is instantiated.
- Illustration of Abstract class:

```
abstract class Shape
{
    int color;
    // An abstract function
    abstract void draw();
}
```

Important observations about abstract classes

- An instance of an abstract class can not be created.
- We can have an abstract class without any abstract method.
- There can be a **final method** in abstract class but any abstract method in class(abstract class) can not be declared as final.
- We can define static methods in an abstract class.
- If a **class** contains at least **one abstract method** then compulsory should declare a class as abstract.
- If the **Child class** is unable to provide implementation to all abstract methods of the **Parent class** then we should declare that **Child class as abstract** so that the next level Child class should provide implementation to the remaining abstract method.
- Elements abstract class can have
  - data member
  - abstract method
  - method body (non-abstract method)
  - constructor
  - main() method.

Example

```
abstract class Sunstar {
    abstract void printInfo();
}
// Abstraction performed using extends
class Employee extends Sunstar {
    void printInfo() {
        String name = "avinash";
        int age = 21;
        float salary = 222.2F;
        System.out.println(name);
        System.out.println(age);
        System.out.println(salary);
    }
}
class Base {
    public static void main(String args[]){
        Sunstar s = new Employee();
        s.printInfo();
    }
}
```

Example: Abstract Class having constructor, data member, and methods

```
import java.io.*;
abstract class Subject {
    Subject() {
      System.out.println("Learning Subject");
    }
      abstract void syllabus();
      void Learn(){
    System.out.println("Preparing Right Now!");
    }
}
class IT extends Subject {
  void syllabus(){
    System.out.println("C , Java , C++");
  }
}
class Test {
    public static void main(String[] args) {
        Subject x=new IT();
            x.syllabus();
            x.Learn();
        }
    }
```

O/P:

```
Learning Subject
C , Java , C++
Preparing Right Now!
```

Example: Abstract *class without any abstract method*

```
abstract class Base {
   // Demo method. This is not an abstract method.
   void fun()
   {
      // Print message if class 1 function is called
      System.out.println("Function of Base class is called");
   }
}
class Derived extends Base {

}
class Main {
public static void main(String args[])
   {
        Derived d = new Derived();
        d.fun();
   }
}
```

Example

```
import java.io.*;
abstract class Demo {
   abstract void m1();
   abstract void m2();
   abstract void m3();
}
abstract class FirstChild extends Demo {
   public void m1() {
     System.out.println("Inside m1");
   }
}
class SecondChild extends FirstChild {
   public void m2() {
     System.out.println("Inside m2");
   }
   public void m3() {
     System.out.println("Inside m3");
   }
}
class Test {
   public static void main(String[] args) {
      // if we remove the abstract keyword from FirstChild Class and
uncommented below obj creation for
      // FirstChild then it will throw compile time error as did't
override all the abstract methods

      // FirstChild f=new FirstChild();
      // f.m1();

      SecondChild s = new SecondChild();
      s.m1();
      s.m2();
      s.m3();
   }
}
```

Practical use cases

- A use case for an abstract class in Java is in a vehicle rental system. The abstract class Vehicle could define common attributes like vehicleID, rentalCost, and methods like startRental() and endRental(), while leaving an abstract method calculateRentalCost() for subclasses like Car, Bike, and Truck to implement based on their own rental cost calculations (e.g., hourly, daily, or based on mileage). This ensures all vehicles follow the same rental flow but handle cost calculations specific to their type.
- In a travel booking system, an abstract class Booking could define common methods like confirmBooking() and cancelBooking(), while leaving an abstract method calculatePrice() to be implemented by subclasses such as FlightBooking, HotelBooking, and CarRentalBooking. Each type of booking would have its own way of calculating the price, but the process of confirming and canceling the booking would remain consistent across all types.

Interface

- The interface in Java is *a* mechanism to achieve abstraction.
- An abstract type used to specify the behavior of a class.
- A Java interface contains static constants and abstract methods.
- Constants: Interfaces can have variables, but they are implicitly public, static, and final.
- An interface cannot contain a constructor (as it cannot be used to create objects)
- A class that implements an interface must provide the concrete implementation for all methods declared in the interface.
- **Multiple inheritance:** A class can implement multiple interfaces, enabling a form of multiple inheritance, which is not possible with classes. // A simple interface

```
interface Player
{
   final int id = 10;
   int move();
}
```

Relationship Between Class and Interface

- A class can extend another class, and similarly, an interface can extend another interface. However, only a class can implement an interface, and the reverse (an interface implementing a class) is not allowed.
- **Difference Between Class and Interface**

Class	Interface
In class, you can instantiate variables and create an object.	In an interface, you must initialize variables as they are final but you can't create an object.
A class can contain concrete (with implementation) methods	The interface cannot contain concrete (with implementation) methods.
The access specifiers used with classes are private, protected, and public.	In Interface only one specifier is used- Public.

- **Note:** To implement an interface, we use the keyword implements.

Example

```
import java.io.*;
interface In1 {
    // public, static and final
    final int a = 10;
    void display();
}
class TestClass implements In1 {
    public void display(){
      System.out.println("Geek");
    }
public static void main(String[] args)
    {
        TestClass t = new TestClass();
        t.display();
        System.out.println(t.a);
    }
}
```

Example

```
import java.io.*;
interface Vehicle {
   void changeGear(int a);
   void speedUp(int a);
   void applyBrakes(int a);
}
class Bicycle implements Vehicle{
   int speed;
   int gear;
   // to change gear
   @Override
   public void changeGear(int newGear){
      gear = newGear;
   }
   // to increase speed
   @Override
   public void speedUp(int increment){
      speed = speed + increment;
   }
// to decrease speed
   @Override
   public void applyBrakes(int decrement){
speed = speed - decrement;
   }
   public void printStates() {
      System.out.println("speed: " + speed
         + " gear: " + gear);
   }
}
class Bike implements Vehicle {
   int speed;
   int gear;
   // to change gear
   @Override
   public void changeGear(int newGear){
      gear = newGear;
   }
   // to increase speed
   @Override
   public void speedUp(int increment){
      speed = speed + increment;
   }
   // to decrease speed
   @Override
   public void applyBrakes(int decrement){
      speed = speed - decrement;
   }
   public void printStates() {
      System.out.println("speed: " + speed
         + " gear: " + gear);
   }
}
class GFG {

   public static void main (String[] args) {
      Bicycle bicycle = new Bicycle();
      bicycle.changeGear(2);
      bicycle.speedUp(3);
      bicycle.applyBrakes(1);

      System.out.println("Bicycle present state
:");
      bicycle.printStates();

      Bike bike = new Bike();
      bike.changeGear(1);
      bike.speedUp(4);
      bike.applyBrakes(3);

      System.out.println("Bike present state
:");
      bike.printStates();
   }
}
```

Static and default methods in interfaces

- From JDK 8, we can now define static and default methods in interfaces.
- Default method: To allow developers to **add new functionality** to interfaces **without forcing all implementing classes to change**. Existing classes still compile fine — they automatically "inherit" the default method.
- Static methods in interfaces are utility or helper methods related to the interface. They **belong to the interface itself**, not to any implementing class.
- Note: Static methods of interfaces are not inherited by implementing classes.

```
interface In1{
   final int a = 10;
   static void display(){
      System.out.println("static method");
   }
}
class TestClass implements In1{
public static void main (String[] args){
      In1.display();
   }
}
```

```
interface In1{
   final int a = 10;
   default void display(){
     System.out.println("default method");
   }
}
class TestClass implements In1{
public static void main (String[] args){
      TestClass t = new TestClass();
      t.display();
   }
}
```

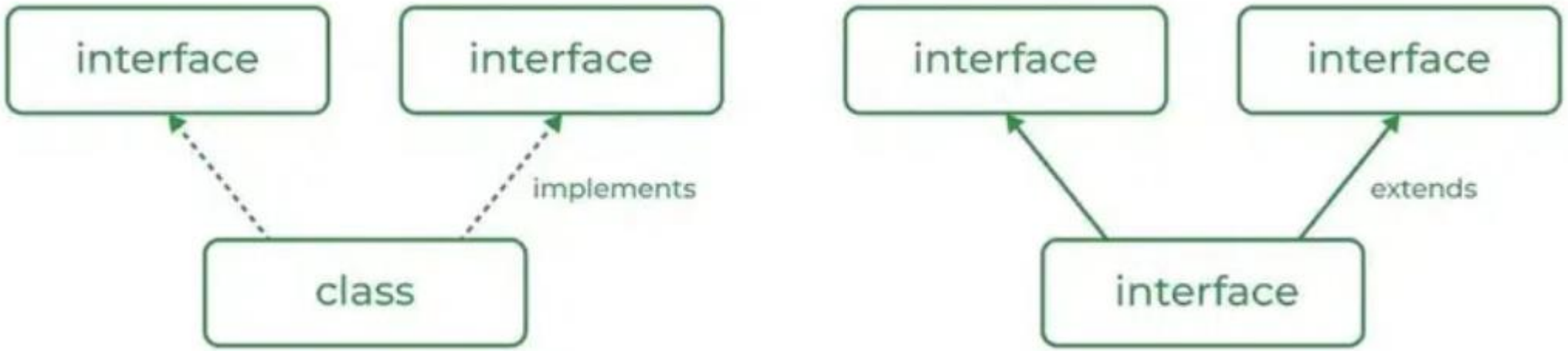
Extending Interfaces

- One interface can inherit another by the use of keyword extends.
- When a class implements an interface that inherits another interface, it must provide an implementation for all methods required by the interface inheritance chain.

//Example:

```
interface A {
   void method1();
   void method2();
}
interface B extends A {
   void method3();
}
        class Test implements B {
           public void method1(){
              System.out.println("Method 1");
           }
           public void method2(){
              System.out.println("Method 2");
              }
            public void method3(){
              System.out.println("Method 3");
              }
}
```

Multiple Inheritance in Java Using Interface



```
interface API {
    // Default method
    default void show()
    {
    System.out.println("Default API");
    }
}

interface Interface1 extends API {
    // Abstract method
    void display();
}

interface Interface2 extends API {
    // Abstract method
    void print();
}

class TestClass implements Interface1, Interface2 {
public void display()
    {
        System.out.println("Display from Interface1");
    }
public void print()
    {
        System.out.println("Print from Interface2");
    }
public static void main(String args[])
    {
        TestClass d = new TestClass();

        d.show(); // Default method from API
        d.display(); // Overridden method from Interface1
        d.print(); // Overridden method from Interface2
    }
}
```

```
// First base interface
interface InterfaceA {
    void methodA();
}

// Second base interface
interface InterfaceB {
    void methodB();
}

// InterfaceC extends InterfaceA and InterfaceB
interface InterfaceC extends InterfaceA, InterfaceB {
    void methodC();
}

// InterfaceD extends InterfaceA and InterfaceB
interface InterfaceD extends InterfaceA, InterfaceB {
    void methodD();
}

// Class implementing InterfaceC and providing
implementations
class ImplementingClass implements InterfaceC {
    @Override
    public void methodA() {
        System.out.println("Method A from InterfaceA");
    }

    @Override
    public void methodB() {
        System.out.println("Method B from InterfaceB");
    }

    @Override
    public void methodC() {
        System.out.println("Method C from InterfaceC");
    }
}

// Main class to test the implementation
public class Main {
    public static void main(String[] args) {
        ImplementingClass obj = new ImplementingClass();
        obj.methodA();
        obj.methodB();
        obj.methodC();
    }
}
```

MCQS

```
class Parent {
    void show() {
        System.out.println("Parent Show");
        special();
    }
    void special() {
        System.out.println("Parent Special");
    }
}
class Child extends Parent {
    void unique() {
        System.out.println("Child Unique");
    }
    @Override
    void special() {
        System.out.println("Child Special");
    }
}
public class Main {
    public static void main(String[] args) {
        Parent obj = new Child();
        obj.show();
    }
}
```

```
abstract class Animal {
    public int age;

    Animal() {
        age = 5;
    }
abstract public void makeSound();
    abstract final public void displayAge();
}
class Dog extends Animal {
    public void makeSound() {
        System.out.println("Woof!");
    }

    final public void displayAge() {
        System.out.println("Age: " + age);
    }
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.makeSound();
        dog.displayAge();
    }
}
```

MCQS

```
class Vehicle {
   void start() {
      System.out.print("Vehicle Starting ");
   }
   void drive() {
      System.out.print("Vehicle Driving ");
   }
}
class Car extends Vehicle {
   void honk() {
      System.out.print("Car Honking ");
   }
   @Override
   void drive() {
      System.out.print("Car Driving ");
   }
}
public class Main {
   public static void main(String[] args) {
      Vehicle obj = new Car();
      obj.drive();
      obj.honk();
   }
}
```

```
public class Main {
   public static void main(String[] args) {
      System.out.println("B" + 5 * 2);
      System.out.println(5 + 2 + "B" + 5 * 2);
   }
}
```

```
public class Main {
   public static void main(String[] args) {
      System.out.println("B" + 5 * 2);
      System.out.println(5 + 2 + "B" + 5 * 2);
      System.out.println("B" + 1 + 4);
   }
}
```

```
 public class Main {
    public static void main(String[] args) {
       String str1 = "Java";
       String str2 = "Java";
       String str3 = new String("Java");
       System.out.println(str1 == str2);
       System.out.println(str1 == str3);
       System.out.println(str1.equals(str3));
    }
 }
```

MCQS

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
class Cat extends Animal {
    void sound() {
        System.out.println("Cat meows");
    }
}
public class DowncastingExample {
    public static void main(String[] args) {
        Animal animal = new Dog();
        Cat cat = (Cat) animal;
        cat.sound();
    }
}
```

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
class Cat extends Animal {
    void sound() {
        System.out.println("Cat meows");
    }
}
public class PolymorphismChallenge {
    public static void main(String[] args) {
        Animal animal1 = new Dog();
        Animal animal2 = new Cat();
        Dog dog = new Dog();
        animal1.sound();
        animal2.sound();
        ((Animal) dog).sound();
    }
}
```

MCQS

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }

}
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
    void diff(){

      System.out.println("DIFF");
    }
}
class Cat extends Animal {
    void sound() {
        System.out.println("Cat meows");
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        Animal animal1 = new Dog();
        Animal animal2 = new Cat();
        Dog dog = new Dog();
        animal1.sound();
        animal2.sound();
        ((Animal) dog).sound();
        ((Animal) dog).diff();
    }
}
```

MCQS

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }

}
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
    void diff(){

      System.out.println("DIFF");
    }
}
class Cat extends Animal {
    void sound() {
        System.out.println("Cat meows");
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        Animal animal1 = new Dog();
        Animal animal2 = new Cat();
        Dog dog = new Dog();
        animal1.sound();
        animal2.sound();
        ((Animal) dog).sound();
        ((Animal) dog).diff();
    }
}
```